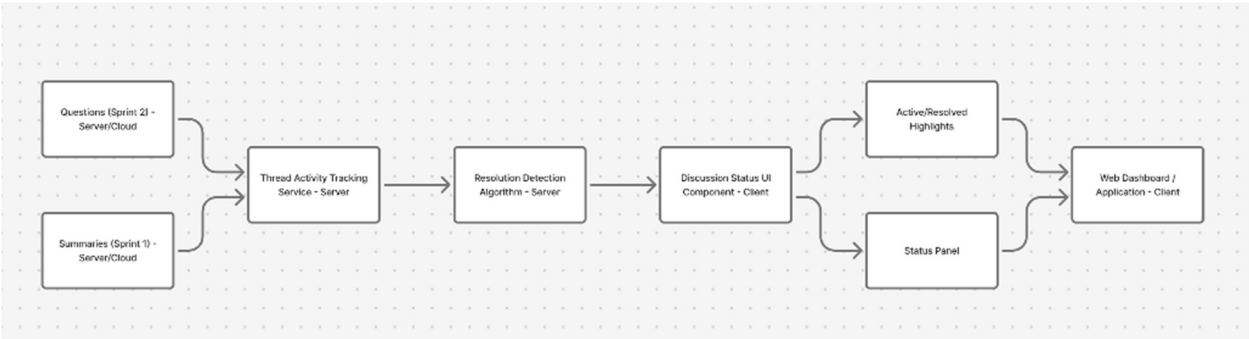
1. Head

Sprint 4 – Active vs Resolved Discussion Status (Context Intelligence)

User Story:
As a user returning from vacation, I want to see which discussions are still active vs. resolved so that I know where my input is still needed. (Independent)

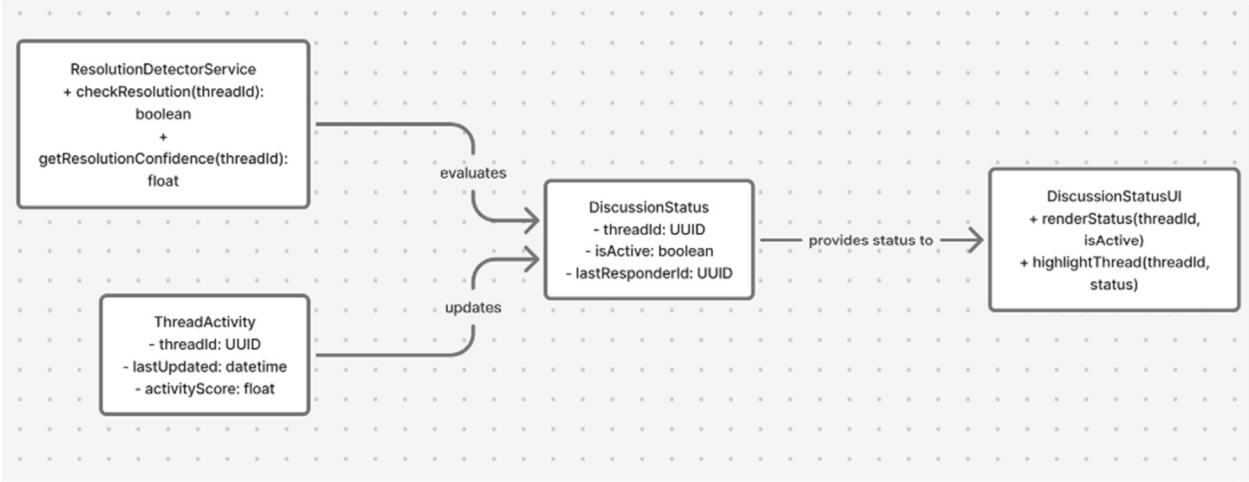
---

2. Architecture Diagram



---

3. Class Diagram



---

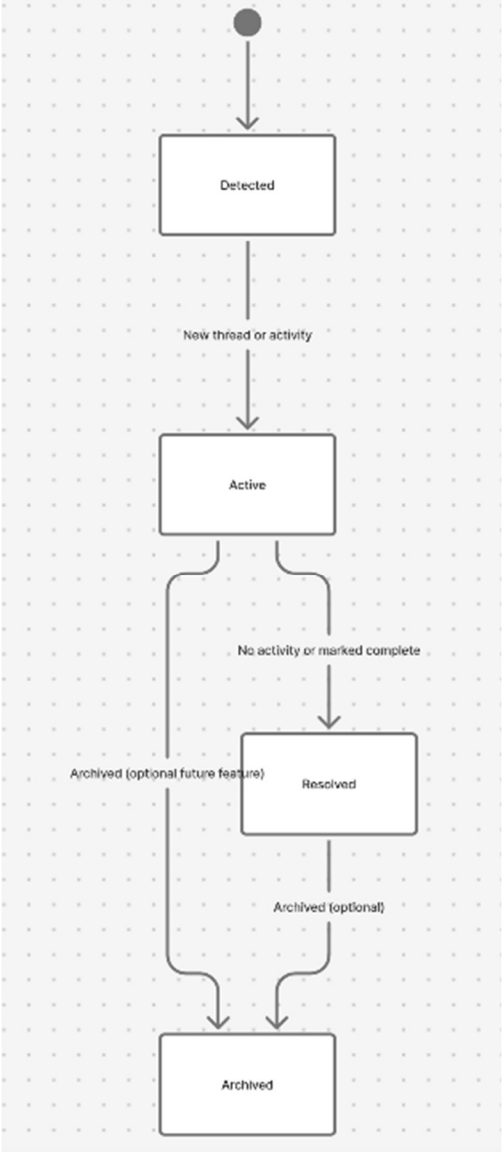
4. List of Classes

- ThreadActivity
- DiscussionStatus

- ResolutionDetectorService
- DiscussionStatusUI

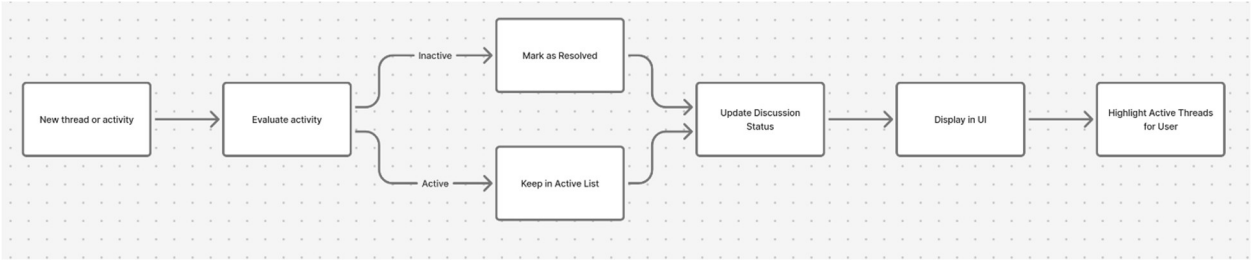
---

5. State Diagrams



---

6. Flow Chart



---

7. Development Risks and Failures

- Algorithm may misclassify discussions as resolved, leading to loss of visibility
- High thread volume could cause performance bottlenecks
- UI updates may lag behind server-side status

---

8. Technology Stack

- Backend: Python / Node.js (server processing)
- Frontend: React / TypeScript (web dashboard)
- Database: PostgreSQL or MongoDB (thread/activity storage)
- Optional: Redis for caching high-frequency thread data

---

9. APIs

- GET /threads/status → Returns list of threads with active/resolved status
- POST /threads/update → Updates thread activity and recalculates status

---

10. Public Interfaces

- Web dashboard components: Discussion Status Panel, Highlights, Active vs Resolved badges
- REST APIs for thread status and updates

---

11. Data Schemas

ThreadActivity

- threadId: UUID – Unique thread identifier
- lastUpdated: datetime – Last activity timestamp
- activityScore: float – Calculated activity level

DiscussionStatus

- threadId: UUID – Unique thread identifier
- isActive: boolean – Active vs resolved flag
- lastResponderId: UUID – Last user to reply

---

12. Security and Privacy

- Only authorized users can see thread status
- Sensitive content is not exposed via summaries or status API
- Data encrypted in transit (HTTPS)

---

13. Risks to Completion

- Integration issues with Sprint 1/2 systems
- Resolution algorithm may not align with user expectations
- UI/UX complexity in effectively displaying thread status